Base de données

1. Moteur

PostgreSQL 15, hébergé sur Supabase. SQLAlchemy 2 en ORM, Alembic pour les migrations, psycopg 3 en pilote.

Deux chaînes de connexion, à prendre dans l'onglet « **Direct Connection string** » du panneau *Connect* — pas l'onglet *ORM*, qui ne propose que Prisma et Drizzle. La distinction entre les deux compte :

Variable	Port	Usage	Pourquoi
`DATABASE_URL`	6543	exécution de l'application	pooler de transactions, supporte beaucoup de connexions courtes
`DIRECT_URL`	5432	migrations Alembic	pooler de sessions ; le pooler de transactions ne supporte pas le DDL

Une troisième chaîne, intitulée *Direct connection* (`db.xxx.supabase.co`), est à éviter : elle n'est joignable qu'en IPv6 et échoue depuis la plupart des connexions domestiques et des exécuteurs d'intégration continue.

Le pooler de transactions ne conserve pas l'état entre deux requêtes : les requêtes préparées côté serveur sont désactivées (`prepare_threshold=None`). Et le paramètre `?pgbouncer=true` que Supabase place dans l'URL est propre à Prisma — libpq le refuse, le serveur le retire automatiquement.

Une base SQLite locale est acceptée pour une démonstration hors ligne : les colonnes JSON sont déclarées `JSONB` avec une variante `JSON` . Ce n'est pas une coquetterie, c'est ce qui permet d'exécuter la suite de tests de bout en bout sans service distant.

2. Conventions

Héritées du schéma Prisma de la première version, et **délibérément conservées** pour qu'une base déjà migrée reste lisible par les deux implémentations.

Élément	Convention	Exemple
Tables	pluriel, `snake_case`	`chat_messages`
Colonnes	`camelCase`	`createdAt` , `promptVersion`
Clés primaires	`id` , UUID v4 en `VARCHAR(36)`	—
Clés étrangères	`<entité>Id`	`courseId`
Horodatages	`createdAt` , `updatedAt` , avec fuseau	—

Côté Python les attributs sont en `snake_case` et pointent vers la colonne `camelCase` (`mapped_column("createdAt", ...)`). Les deux lectures restent donc naturelles.

3. Les huit tables

```
users
  ├──< courses
  │      ├──< summaries
  │      ├──< quizzes ──< quiz_attempts
  │      └──< chat_messages
  └──< exams

benchmark_runs   (indépendante)
```

`exams` pend de l'**étudiant**, pas d'un cours : un examen blanc porte sur plusieurs supports, et supprimer l'un d'eux ne doit pas emporter l'examen qui portait aussi sur les trois autres.

`users`

L'étudiant. Pas de rôle : StudyAI n'a qu'un type d'utilisateur.

Colonne	Type	Note
`id`	VARCHAR(36)	UUID
`email`	VARCHAR(255)	**unique**, indexé
`passwordHash`	VARCHAR(255)	bcrypt, jamais le mot de passe
`createdAt`	TIMESTAMPTZ	

`courses`

Un cours importé : texte collé ou PDF à couche texte.

Colonne	Type	Note
`ownerId`	VARCHAR(36)	→ `users.id`, `ON DELETE CASCADE`
`title`	VARCHAR(300)	
`subject`	VARCHAR(200)	facultatif
`content`	TEXT	**traverse l'anonymisation avant tout envoi**
`charCount`	INTEGER	affiché, et sert à anticiper le découpage
`hash`	VARCHAR(64)	empreinte SHA-256 tronquée du contenu
`createdAt` / `updatedAt`	TIMESTAMPTZ	

Index : `(ownerId, createdAt)` — la liste des cours d'un étudiant, triée par date, est la requête la plus fréquente.

`summaries` — module 1

Colonne	Type	Note
`courseId`	VARCHAR(36)	→ `courses.id`, cascade
`payload`	JSONB	sortie validée : `{abstract, keyPoints[], keywords[], warnings[]}`
`level`	VARCHAR(20)	`court` / `standard` / `detaille`
`model`	VARCHAR(100)	identifiant exact du modèle ayant répondu
`promptVersion`	VARCHAR(20)	
`temperature`	NUMERIC(3,2)	
`tokensIn` / `tokensOut`	INTEGER	
`latencyMs`	INTEGER	
`sentPayload`	TEXT	**la charge exacte envoyée au modèle**
`segmentCount`	INTEGER	1 = cours court, n = agrégation
`tokenTypes`	JSONB	types de données réellement masquées

Le couple `(courseId, level)` sert de clé de cache : un même cours au même niveau n'est pas réanalysé.

`sentPayload` et `tokenTypes` alimentent la vue de transparence. Ils sont coûteux en place — le cours y est recopié — et c'est assumé : c'est la seule preuve vérifiable de l'anonymisation.

`quizzes` — module 2

Mêmes colonnes de télémétrie. Spécifiques :

Colonne	Type	Note
`payload`	JSONB	`{questions: [{question, options[4], correctIndex, explanation, sourceQuote}]}`
`questionCount`	INTEGER	nombre réellement produit
`difficulty`	VARCHAR(20)	`facile` / `standard` / `difficile`

`correctIndex`, `explanation` et `sourceQuote` restent en base. L'API les retire avant d'envoyer le quiz au navigateur ; sans quoi l'inspecteur réseau suffirait à tricher.

`quiz_attempts`

Colonne	Type	Note
`quizId`	VARCHAR(36)	→ `quizzes.id`, cascade
`answers`	JSONB	indices choisis, `-1` = sans réponse
`score` / `total`	INTEGER	calculés **côté serveur**

`chat_messages` — module 3

Colonne	Type	Note
`role`	VARCHAR(20)	`user` ou `assistant`
`content`	TEXT	
`sources`	JSONB	extraits du cours cités ; vide si hors cours
`grounded`	BOOLEAN	faux = le cours ne permettait pas de répondre
télémétrie		`model`, `promptVersion`, jetons, latence — nuls pour un message d'étudiant

La question de l'étudiant est enregistrée **avant** l'appel au modèle : si l'appel échoue, le fil reste intact et il peut relancer.

`exams` — examen blanc multi-cours

Colonne	Type	Note
`id`	VARCHAR(36)	UUID
`ownerId`	VARCHAR(36)	→ `users.id`, cascade. **Pas de `courseId`**
`courses`	JSONB	`[{id, title}]` au moment de la composition. Conservé pour que l'historique reste lisible après la suppression d'un cours
`questions`	JSONB	les questions **figées**, avec leur réponse et leur cours d'origine
`warnings`	JSONB	ce que la composition n'a pas pu équilibrer, et pourquoi
`durationS`	INTEGER	temps accordé. Par défaut 60 s par question
`startedAt`	TIMESTAMPTZ	le serveur fait foi sur le temps écoulé
`submittedAt`	TIMESTAMPTZ	`NULL` tant que la copie n'est pas rendue. Non nul ⇒ **plus modifiable**
`answers`	JSONB	`null` pour une question laissée vide — cas normal sur un examen chronométré
`score` / `total`	INTEGER	
`late`	BOOLEAN	copie rendue hors délai. **Corrigée quand même**
`createdAt`	TIMESTAMPTZ	

Index : `(ownerId, createdAt)`.

Pourquoi les questions sont dupliquées ici plutôt que référencées par `(quizId, rang)` : un quiz peut être régénéré ou supprimé, et un examen déjà passé doit rester corrigible exactement tel qu'il a été passé. La normalisation cède devant la reproductibilité — voir DECISIONS.md 018.

`benchmark_runs` — support du banc d'essai (AA5)

Colonne	Type
`task`	`summarize` / `quiz` / `chat`
`corpusVersion`, `promptVersion`, `model`	VARCHAR
`metrics`	JSONB

Table créée mais pas encore alimentée. C'est le lot de Khalil.

4. Intégrité

Toutes les relations sont en `ON DELETE CASCADE`. Supprimer un cours supprime ses résumés, ses quiz, les tentatives associées et le fil de discussion. L'étudiant en est averti par une confirmation explicite dans l'interface.

5. Données sensibles

Donnée	Traitement
Mot de passe	bcrypt, jamais stocké en clair, jamais journalisé
Contenu du cours	stocké brut — c'est la propriété de l'étudiant ; **anonymisé avant envoi au modèle**, pas avant stockage
`sentPayload`	déjà anonymisé par construction
Clés d'API	jamais en base, uniquement dans `.env`, non versionné

La table des jetons (`[EMAIL_1]` → valeur réelle) vit **en mémoire le temps d'une requête** et n'est jamais persistée.

6. Migrations

Révision	Objet
`0001`	schéma initial, 7 tables
`0002`	colonne `tokenTypes` sur `summaries`

Règles : toute modification du schéma passe par une migration, qui doit produire du SQL valide hors ligne (`alembic upgrade head --sql`). La CI le vérifie à chaque poussée.

```
cd server
python -m alembic upgrade head        # appliquer
python -m alembic upgrade head --sql  # inspecter sans se connecter
```

7. Données d'amorçage

Il n'y en a pas, volontairement. Les supports de cours du module (`Seance 5/Atelier 5.pdf` , `Seance 2/Prompt Engineering.pdf`) servent de jeu d'essai : un corpus réel, que l'enseignant reconnaîtra en soutenance.